



ارائه به تیم فنی

پیاده‌سازی فنی بلاک‌چین سور

شبکه‌ی Besu، قراردادهای هوشمند، سرویس‌های آف‌چین و روش راه‌اندازی

مهر ۱۴۰۵

دستور جلسه

۱ سور و واژگان این ارائه

۲ شبکه‌ی Besu و اجماع

۳ قراردادهای ساختاری و روش استقرار در genesis

۴ قراردادهای یک‌به‌یک

۵ توزیع پاداش، سقف و حکمرانی نرخ

۶ کلیدها و سرویس‌های آف‌چین

۷ زیرساخت، بازیابی اضطراری و امنیت

۸ آزمون‌ها، یافته‌های عملیاتی و موارد باز

سور چیست و واژگان این ارائه

سور یک بلاک چین عمومی و بدون مجوز برای توسعه دهندگان و کاربران ایرانی است. شبکه روی Hyperledger Besu رسمی (بدون فورک) با اجماع QBFT اجرا می شود و ارز بومی آن «سورن» است. تقسیم پاداش، حکمرانی و عضویت ولیدیتورها همگی در قراردادهای هوشمند پیاده شده اند، نه در نرم افزار زنجیره.

ولیدیتور

گره ای که بلاک تولید و تأیید می کند؛ مجموعه ی آن از قرارداد ValidatorsRegistry خوانده می شود

سورن

ارز بومی زنجیره (مثل ETH در اتریوم)، نه توکن ERC20؛ کوچک ترین واحد wei است و ۱۸ رقم اعشار دارد

ریوارد و فی

ریوارد: ۲ سورن ثابت برای هر بلاک؛ فی: کارمزد تراکنش ها. هر دو به آدرس ثابت BlockRewardDistributor می رسند

genesis و alloc

بلاک نخست زنجیره؛ بخش alloc فایل genesis کد و storage قراردادهای موجودی های اولیه را مستقیم می نویسد

اوراکل

کلید یک سرویس آف چین که یک نوع داده ی بیرونی (توزیع پاداش، وضعیت ولیدیتور، هویت، پرداخت) را روی زنجیره ثبت می کند

epoch توزیع

یک دور تسویه ی پاداش (حداقل هر ۲۳ ساعت) که بازه ای از بلاک ها را پرداخت می کند؛ «N-1» یعنی وضعیت قرارداد در انتهای بلاک قبل از N

معماری کلی: زنجیره، شش قرارداد، چهار سرویس آفچین

لایه‌ی زنجیره

Hyperledger Besu رسمی؛ QBFT با حالت انتخاب ولیدیتور از قرارداد (validatorselectionmode: contract)؛ بلاک ۳ ثانیه؛ نهایی‌شدن فوری؛ ریوارد و فی مستقیم به قرارداد توزیع می‌رسد.

IdentityRegistry
0x6666

ValidatorsTreasury
0x5555

ValidatorsBoard
0x4444

ValidatorsRegistry
0x3333

BlockRewardDistributor
0x2222

FoundationDAO
0x1111

قراردادهای ساختاری (در genesis، بدون constructor)

PaymentReporter
کلید: paymentOracle

Identity Service
کلید: identityOracle

Verifier
کلید: verifier

RewardRouter
کلید: distributionOracle

سرویس‌های آفچین با چهار کلید مستقل؛ هرکدام فقط یک نوع تصمیم را روی زنجیره ثبت می‌کند

پیکربندی شبکه: genesis و نود

پارامتر	مقدار	توضیح
کلاینت	Besu رسمی (آزموده: ۲۶/۷/۱ و ۲۶/۹/۰)	بدون فورک؛ نسخه‌ی ۲۶/۹/۰ به Java 25 نیاز دارد
اجماع	QBFT، حالت contract	validatorcontractaddress = 0x3333... و qbft.validatorselectionmode = contract 3333
blockperiodseconds	۳	آهنگ مشاهده‌شده حدود ۲/۹۵ ثانیه است؛ شمار بلاک را از زمان حدس نزنید
epochlength / requesttimeoutseconds	۱۰ / ۳۰۰۰۰	هر ولیدیتور بدون نود واقعی در هر نوبتش به‌اندازه‌ی requesttimeout تأخیر می‌سازد
miningbeneficiary و blockreward	0x2222...2222 و wei ۱۰ ¹⁸ ×۲	پیش‌فرض blockreward صفر است؛ اگر نوشته نشود هیچ ریواردی صادر نمی‌شود
min-gas-price و zeroBaseFee	wei ۱۰ ¹⁴ و true	genesis min-gas-price تنظیم سطح نود است، نه فیلد genesis
hard fork	همه از بلاک ۰ تا london	اجباری؛ evmVersion کامپایل باید با فورک genesis هم‌خوان باشد (PUSH0 فقط از Shanghai)
extraData	RLP با ۰ ولیدیتور	در حالت contract، فهرست ولیدیتور فقط از قرارداد می‌آید، نه از extraData

انتخاب ولیدیتور از قرارداد و خطر ولیدیتور بدون نود

زمان اثر تغییر



تغییر فهرست بدون تأخیر epoch و بدون کش اعمال می‌شود. دقیقاً از کدام بلاک اثر می‌کند را روی نسخه‌ی واقعی Besu اندازه بگیرید؛ سیاست پرداخت پاداش از وضعیت انتهای بلاک N-1 استفاده می‌کند.

رابط قرارداد



Besu هر بلاک تابع `getValidators()` را از `ValidatorsRegistry` می‌خواند؛ امضای آن در کد Besu ثابت است. خروجی: فهرست آدرس‌های وضعیت `Active`.

بازگشت از قفل



اگر زنجیره قفل شد، همه‌ی نودها را هم‌زمان ری‌استارت کنید؛ ری‌استارت جزئی معمولاً جواب نمی‌دهد چون تایمر دور نودهای زنده جلوتر رفته. آخرین راه، گذار blockheader است (اسلاید بازیابی اضطراری).

ولیدیتور بدون نود



نصاب QBFT برابر دوسوم n (گرد به پایین) به‌علاوه یک است. فعال‌کردن ولیدیتوری که نود واقعی پشتش نیست، اگر نصاب از شمار نودهای روشن بیشتر شود، کل زنجیره را قفل می‌کند (در آزمون واقعی حدود ۵۴ دقیقه). پس شمار نود روشن همیشه $\leq$ ولیدیتور فعال.

شش قرارداد ساختاری: آدرس ثابت و وابستگی‌ها

قرارداد	آدرس	نقش	قراردادهایی که می‌شناسد
FoundationDAO	0x1111...1111	حکمرانی داخلی بنیاد (۱۵ عضو) و کنترل دو اوراکل	هیچ‌کدام
BlockRewardDistributor	0x2222...2222	دریافت ریوارد و فی؛ توزیع؛ سقف و حکمرانی نرخ	Registry, Treasury, Board
ValidatorsRegistry	0x3333...3333	مرجع اجماع، وثیقه، چرخه‌ی عمر و جریمه	Treasury
ValidatorsBoard	0x4444...4444	هیأت ۵ صندلی با اختیارات تفویضی	Registry, Distributor, Treasury, IdentityRegistry
ValidatorsTreasury	0x5555...5555	خزانه؛ خرج فقط با هیأت و زیر سقف‌ها	Registry, Board
IdentityRegistry	0x6666...6666	هویت خوداظهاری و وریفای همه‌ی کاربران	FoundationDAO

آدرس‌ها پیش از وجود هر قرارداد معلوم‌اند؛ پس هرکدام آدرس `constant` می‌داند و مرحله‌ی سیم‌کشی پس از استقرار وجود ندارد. SurenSale و ServiceStaking قراردادهای مستقل‌اند و با تراکنش عادی مستقر می‌شوند.

چرا قراردادهای در alloc تزریق می‌شوند و نه با تراکنش

مقدار اولیه اجرا نمی‌شود



مقداردهی scalarها (آدرس اوراکل‌ها، سقف‌ها، پارامترهای زمانی، سهم‌ها) جزو runtime نیست. هر scalar غیرصفر باید با overlay صریح روی slot خودش نوشته شود، وگرنه صفر می‌ماند.

بدون constructor



کد (runtime) و storage نهایی مستقیم در alloc نوشته می‌شود. هر شش قرارداد constructor ندارند، چون قراردادهای genesis با constructor روی اکسپلوررهای مثل Blockscout وریفای نمی‌شوند.

بدون vialR، بدون immutable



کامپایل با optimizer، solc 0.8.24، فعال، بدون vialR (برای وریفای روی اکسپلورر). هیچ قرارداد genesis متغیر immutable ندارد؛ پس برداشتن مستقیم deployedBytecode درست است.

mapping و آرایه



این‌ها با keccak محاسبه می‌شود. سه قرارداد دارای عضو اولیه (FoundationDAO، Registry، Board) با یک قرارداد کمکی موقت روی زنجیره‌ی محلی اجرا و storage واقعی از state خوانده می‌شود، نه محاسبه‌ی دستی.

ابزار ساخت genesis.json: دو مسیر برای دو نوع قرارداد

مسیر شبیه‌سازی: FoundationDAO، Registry، Board

۱. جایگزینی آدرس‌های اولیه در قرارداد کمکی (بدون آرگومان constructor)
۲. دیپلوی روی زنجیره‌ی محلی موقت (Anvil) و گرفتن state با anvil_dumpState
۳. برداشتن storage؛ کد از قرارداد اصلی، نه کمکی
۴. برای Registry علاوه بر آن scalar overlayها

مسیر ساده: Distributor، Treasury، IdentityRegistry

۱. کامپایل سورس با مقدارهای واقعی
۲. اجرای creation code روی زنجیره‌ی موقت و خواندن کد با eth_getCode
۳. خواندن شماره‌ی slot هر scalar از storageLayout
۴. نوشتن مقدار واقعی در همان slot در alloc.storage

مقدارهایی که باید با overlay نوشته شوند (چون اجرای مقداردهی اولیه جزو runtime نیست)

قرارداد	مقدارها
ValidatorsRegistry	genesis.timestamp؛ paidValidatorCount را ننویسید. verifier؛ entryThresholdBase؛ growthFactorPerValidator؛ membershipFeeBps؛ شش پارامتر امنیتی و زمانی؛ windowStart (برابر)
ValidatorsTreasury	periodCap و perPaymentCap
BlockRewardDistributor	validatorDirectShareBps و distributionOracle

بررسی‌های اجباری پس از ساخت: همه از بلاک صفر

بررسی	مقدار مورد انتظار
Registry: getValidators، isValidator (مؤسس)	فهرست مؤسسان؛ true
Registry: everActivated (مؤسس)	true؛ بدون آن اولین توزیع با «revert» address was never a legitimate validator می‌شود
Registry: wasActiveAt (مؤسس، 0)، statusNonce	true و (برای غیرمؤسس false)
Registry: paidValidatorCount، isPaidEntrant (مؤسس)	و false
Registry: verifier، پارامترها، windowStart	برابر پیکربندی؛ windowStart = timestamp genesis
Treasury: perPaymentCap، periodCap	۵۰,۰۰۰ و ۲۰۰,۰۰۰ سون (صفر یعنی هیچ پرداختی ممکن نیست)
Distributor: lastSettledBlock، rewardRateChangeCount، rateProposalCount	هر سه ۲۰۰۰؛ maxRewardsForRange(1, 1000) = ۲۰۰۰ سون؛ بدون immutable
Board و DAO: getBoardMembers، boardVersion، lastBoardRefreshAt، membershipNonce	۵ عضو؛ مقدار overlay؛ timestamp genesis؛

حالت verify: یک نود Besu واقعی با همان genesis نهایی بالا می‌آید و مقدارهای بالا با eth_call خوانده می‌شود؛ هر ناهمخوانی خروجی غیرصفر و پیام دقیق می‌دهد. ورودی صفر یا تکراری، هیأتی غیر از ۵ عضو و timestamp نامعتبر پیش از ساخت رد می‌شوند.

ValidatorsRegistry: توابع اصلی

تابع	چه کسی	کار
<code>requestMembership()</code> payable	هر آدرس	<code>msg.value</code> = وثیقه + کارمزد؛ سقف نرخ ورود؛ کارمزد به Distributor می‌رود؛ وضعیت Probation
<code>recordActivation(addr, evidenceHash)</code>	verifier	پس از <code>probationPeriod</code> و موفقیت آف‌چین $\leq 95\%$ ؛ آدرس وارد <code>getValidators</code> می‌شود
<code>recordSuspension(addr, evidenceHash)</code>	verifier	حذف فوری از مجموعه (Demoted)؛ مبلغ جریمه <code>slashBps</code> از وثیقه در همان لحظه ثبت می‌شود
<code>recordRecovery</code> ، <code>recordPreExitViolation</code>	verifier	بازگشت پس از <code>recoveryPeriod</code> بدون پرونده‌ی باز؛ یا ثبت تخلف پیش از خروج تا ۷۲ ساعت پس از درخواست خروج
<code>requestExit()</code> و <code>withdrawStake()</code>	ولیدیتور	خروج فوراً وظیفه را پایان می‌دهد؛ برداشت پس از <code>exitCooldown</code> ؛ با پرونده‌ی باز فقط مبلغ درگیر نگه داشته می‌شود
<code>proposeParameterChange</code> ، <code>voteParameterChange</code>	ولیدیتور Active	شش پارامتر امنیتی؛ نصاب اکثریت ساده‌ی Active‌های هنگام ساخت؛ اجرا در همان رأی نصاب
<code>setEntryThresholdBase</code> ، <code>setGrowthFactorPerValidator</code> ، <code>setMembershipFeeBps</code> ، <code>setVerifier</code>	فقط Board	پس از رأی هیأت؛ سه پارامتر اقتصادی با حدود مجاز و قفل مشترک ۱۸۰ روزه
<code>getValidators</code> ، <code>isValidator</code> ، <code>everActivated</code> ، <code>wasActiveAt</code> ، <code>getValidatorInfo</code> ، <code>currentEntryThreshold</code>	view	<code>receive()</code> عمداً <code>revert</code> می‌کند تا موجودی قرارداد برابر جمع وثیقه‌های قفل‌شده بماند

ValidatorsRegistry: منحنی وثیقه و پارامترها

پارامتر امنیتی (ParamKey)	مقدار	قید
MaxEntriesPerWindow / EntryWindowSeconds	۱ / ۸۶۴۰۰ ث	—
ProbationPeriod	۶۰۴۸۰۰ ث (۱ هفته)	—
RecoveryPeriod	۱۷۲۸۰۰ ث (۴۸ ساعت)	بیش از ۳۶۰۰ ث
SlashBps	۱۰۰ (۱٪)	$10,000 \geq$
ExitCooldown	۶۰۴۸۰۰ ث (۱ هفته)	بیش از ۷۲ ساعت

منحنی عمداً برای ۵۰ تا ۱۰۰ ولیدیتور طراحی شده است (وثیقه حدود ۱/۰۵ تا ۲/۵۱ میلیون سورن برای ۵۰ تا ۱۰۰ ولیدیتور در مجموع، با هفت مؤسس رایگان و بدون خروج)؛ بیرون از این بازه سریع گران می‌شود.

فرمول وثیقه (۱۸ fixed-point رقمی)

$$\text{threshold}(p) = \text{entryThresholdBase} \times g^p$$

$p = \text{paidValidatorCount}$ ؛ مؤسسان رایگان شمرده نمی‌شوند

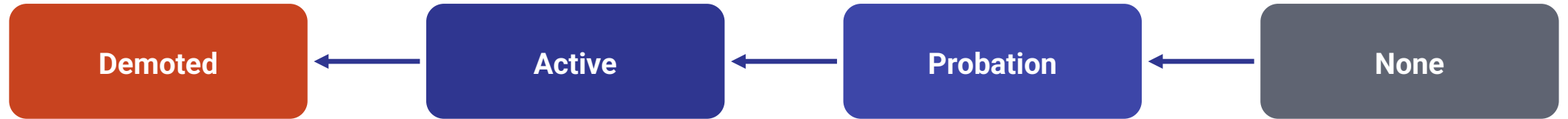
$$g = 1/017479692102686336 \text{ (دوبرابر در هر ۴۰ نفر)؛ توان با } \text{exponentiation-by-squaring}$$

$$\text{کارمزد عضویت} = \text{threshold} \times \text{membershipFeeBps} \div 10,000 = (\text{۴۰۰ یعنی } 4\%)$$

پارامتر اقتصادی (فقط با هیأت)	مقدار اولیه	حد مجاز
entryThresholdBase	۵۰۰,۰۰۰ سورن	۱۰۰,۰۰۰ تا ۲,۰۰۰,۰۰۰
growthFactorPerValidator	دوبرابر هر ۴۰ نفر	دوبرابر هر ۲۰ تا ۸۰ نفر
membershipFeeBps	۴۰۰	۱۰۰ تا ۱۰۰۰

پس از هر تغییر موفق در هرکدام از این سه پارامتر، تا ۱۸۰ روز هیچ‌کدام از سه پارامتر قابل‌تغییر نیست (قفل مشترک).

Registry: ماشین حالت و مسیر پرونده‌ی تعلیق



recordRecovery با Demoted → Active؛ بازگشت از Exiting، Probation یا Active به Demoted

حذف فوری	۱	recordSuspension ولیدیتور را بی‌قید از <code>getValidators</code> خارج می‌کند؛ این مرحله هرگز معطل نمی‌ماند.
رخداد جمعی	۲	اگر در پنجره‌ی ثابت ۱ ساعته بیش از ۲۰٪ مجموعه معلق شوند، همه معاف می‌شوند؛ <code>resolveMassFailureCheck</code> برای هر پرونده جدا.
تحويل شواهد	۳	<code>confirmDelivery</code> فقط دریافت را تأیید می‌کند. اگر تأیید نشد، مجمع روی «آیا تحويل شد» رأی می‌دهد.
اعتراض	۴	ظرف ۷۲ ساعت از تحويل (فقط خود ولیدیتور)؛ مجمع با اکثریت ساده، مهلت ۷ روز؛ نبود نصاب یعنی رد جرمه.
جریمه‌ی بی‌اعتراض	۵	<code>executeUncontestedSlash</code> پس از ۷۲ ساعت؛ مبلغ همان مبلغ ثبت‌شده در لحظه‌ی پرونده (<code>decisionSlashAmount</code>) و به Treasury می‌رود.

ValidatorsBoard و ValidatorsTreasury

ValidatorsTreasury

Board قرارداد `boardApproveExpenditure(to, amount, description)`: فقط قرارداد Board (پس از $3 \leq \text{رأی}$). شرط: $\text{amount} \leq \text{perPaymentCap}$ (۵۰,۰۰۰ سورن، مقدار مرزی مجاز) و مجموع پنجره‌ی حدود ۳۰ روزه (سطل‌های روزانه) $\text{periodCap} \geq (200,000)$.
 هر پرداخت، با هر مقصد و توضیح، در یک شمارنده‌ی مشترک حساب می‌شود (خردکردن راه دورزدن نیست).
`proposeCapChange(kind, value)` و `voteCapChange(id)`: ولیدیتور Active؛ با رسیدن به نصاب، تغییر در صف تأخیر ۷ روزه می‌رود.
`applyPendingCapChange(kind)`: هرکس، پس از ۷ روز؛ قبل از آن `revert` با `timelock not elapsed`.
`receive()`: از `Distributor` (سهم باقی‌مانده‌ی ریوارد) و `Registry` (جریمه). موجودی در `genesis` ندارد؛ سقف‌ها `overlay` می‌شوند.

۵ – ValidatorsBoard صندلی

`voteFor(candidate) / unvoteFor`: هر ولیدیتور Active با هویت ثبت‌شده تا ۵ کاندیدای هم‌زمان؛ کاندیدا باید Active باشد؛ خودرأیی مجاز.
`refreshBoard()`: بدون مجوز، حداکثر هر ۳۰ روز. عضو Active فعلی کرسی‌اش را حفظ می‌کند؛ کرسی خالی با پررأی‌ترین واجد پر می‌شود؛ جای عضو فعلی را فقط نامزدی با رأی اکیداً بیشتر از ضعیف‌ترین عضو می‌گیرد.
`hasBoardAuthority`: کرسی دارد و درخواست خروج نداده؛ تعلیق اختیار را قطع نمی‌کند. `syncBoard()` و `fillVacancies()` بدون مجوزند.
 اکشن‌ها (چرخش اوراکل و `verifier`، تأیید پرداخت، سه پارامتر اقتصادی) با `voteAction`؛ نصاب اکثریت (تأیید پرداخت: $3 \leq \text{رأی}$). اکشنی که با `boardVersion` قدیمی ساخته شده `revert` می‌شود.
 اگر دارنده‌ی کرسی درخواست خروج داده، همه‌ی اکشن‌ها تا `revert syncBoard()` می‌کنند.

BlockRewardDistributor.distributeRewards: ترتیب کنترل‌ها و پرداخت

`distributeRewards(BlockRange{fromBlock, toBlock}, address[] validators, uint256[] blocksMined, uint256 totalRewards, uint256 totalFees)`

فقط distributionOracle می‌تواند آن را صدا بزند و تابع nonReentrant است

۱	فهرست غیرخالی و هم‌طول	validators و blocksMined باید هم‌طول و غیرخالی باشند.
۲	فاصله‌ی ۲۳ ساعت	پس از اولین توزیع، حداقل ۲۳ ساعت از توزیع قبلی.
۳	پیش‌محاسبه	جمع بلاک‌ها؛ کارمزد عضویت انباشته به استخر کارمزد افزوده می‌شود؛ ۳۰٪ کارمزد معمولی برای سوزاندن؛ موجودی کافی؛ مجموع بلاک‌ها صفر نباشد.
۴	پیوستگی بازه	$fromBlock = lastSettledBlock + 1$ ؛ $toBlock \geq fromBlock$ و $block.number >$ ؛ جمع $blocksMined \geq$ اندازه‌ی بازه.
۵	سقف پاداش	$totalRewards \leq maxRewardsForRange(from, to)$ ؛ در غیر این صورت «totalRewards exceed approved reward for range».
۶	پرداخت هر ولیدیتور	آدرس‌ها اکیداً صعودی و غیرصفر، everActivated=true؛ سهم پاداش و سهم کارمزد در یک انتقال: $[mبلغ \times totalBlocks \div blocks]$.
۷	بنیاد، خزانه، سوزاندن	۱۵٪ ریوارد به FoundationDAO؛ باقی‌مانده و خرده‌ی گردکردن به Treasury؛ ۳۰٪ کارمزد معمولی به $address(0)$ ؛ سپس ثبت epoch و $lastSettledBlock = toBlock$.

ساختار داخلی: `prepareEpoch_` (نتیجه در `EpochPrep` struct)، `settleRange_`، شرط سقف، `_payOneValidator + payValidators` (پارامترها در `EpochContext` struct) و `_finalizeEpoch`؛ کامپایل بدون `viaIR`.

بلاک‌چین سور – ارائه به تیم فنی

حکمرانی سهم ولیدیتورها و نرخ پاداش در Distributor

نرخ پاداش هر بلاک

`proposeRateChange(startBlock, ratePerBlock)`: ولیدیتور `Active`؛ `startBlock` باید آینده، حداقل `۲۰۱,۶۰۰` بلاک بعد از بلاک اجرا، و بعد از آخرین نرخ مصوب باشد (در ساخت و در اجرا).

`boardVoteRateChange` و `validatorVoteRateChange`: مهلت رأی `۳۰` روز؛ هنگام تکمیل هر دو مجلس نسخه‌ی هیأت دوباره چک می‌شود.
`executeRateChange(id)`: هرکس، ≤ ۷ روز پس از تکمیل (قبل از آن: `execution delay has (not elapsed)`).

• `rateChangeStatus(id) → (status, problem)`: `۱` رأی‌گیری، `۲` منقضی، `۳` اجراشده، `۴` منتظر تأخیر، `۵` قابل اجرا، `۶` تصویب‌شده ولی غیرقابل اجرا، `۷` تصویب‌نشده و غیرقابل ادامه. `۱`: `problem` ترکیب هیأت، `۲` ارتفاع آینده نیست، `۳` فاصله $> ۲۰۱,۶۰۰$ ، `۴` ترتیب صعودی.

تاریخچه‌ی `rewardRateChanges` (`۲۴` storage slot تا `۲۸`) فقط افزودنی؛ `rateProposals(id)` یازده خروجی دارد.

سهم مستقیم ولیدیتورها (`validatorDirectShareBps`)

`proposeShareChange(bps)`: فقط ولیدیتور `Active`؛ بازه‌ی مجاز `۴۰۰۰` تا `۶۵۰۰` (شروع `۵۰۰۰`).

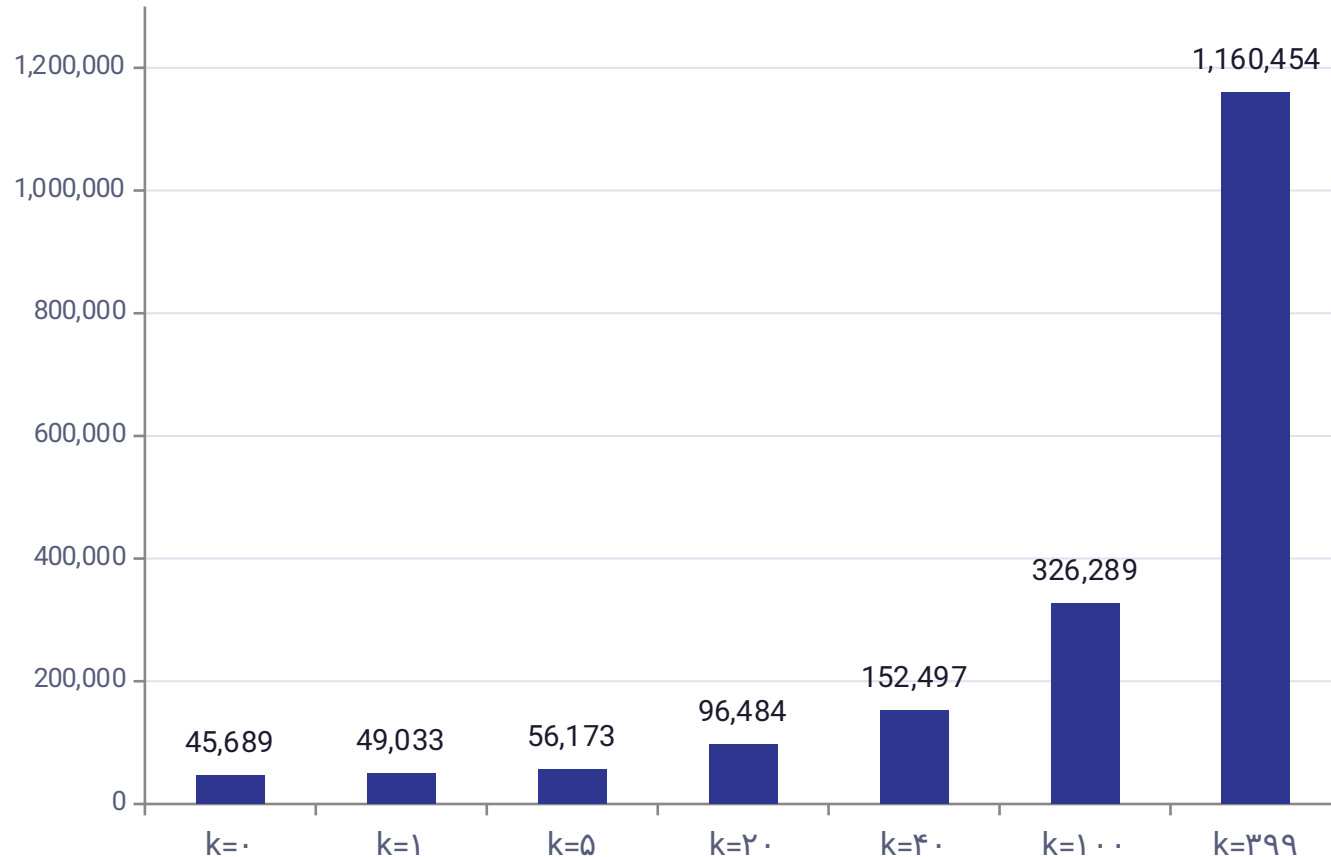
`boardVoteShareChange(id): hasBoardAuthority`: `boardVersion` + زنده هنگام ساخت؛ نصاب `۳` از `۵`.

`validatorVoteShareChange(id)`: نصاب $(۲n+۲) \div ۳$ با `n` = شمار `Active` هنگام ساخت (منجمد)؛ واجد بودن هنگام ساخت.

رأیی که هر دو مجلس را کامل کند، همان‌جا اجرا می‌کند. فاصله‌ی `۱۸۰` روز بین دو تغییر موفق هم هنگام ساخت و هم هنگام اجرا چک می‌شود؛ رأی کامل‌کننده‌ی زودهنگام `revert` می‌شود (حالت `state` رأی برمی‌گردد). انقضا `۳۰` روز.

نسخه‌ی ترکیب هیأت در رأی هیأت و در اجرا دوباره بررسی می‌شود.

محاسبه‌ی سقف پاداش: هزینه‌ی گس



گس maxRewardsForRange؛ تاریخچه‌ی ۴۰۰ ورودی؛ k = شمار تغییر نرخ داخل بازه

جست‌وجوی دودویی

تاریخچه‌ی نرخ فقط افزودنی و صعودی است؛ نرخ لازم برای fromBlock با جست‌وجوی دودویی پیدا می‌شود ($O(\log n)$) و سپس k تغییر داخل بازه جمع می‌شود ($O(k)$).

اثر بر هزینه

بازهی بدون تغییر با تاریخچه‌ی ۴۰۰ ورودی ۴۵,۶۸۹ گس است؛ پیمایش خطی همین حالت ۱,۰۷۲,۷۷۷ گس می‌شد. برای $k \geq 20$ حدود ۲,۸۰۰ گس به‌ازای هر تغییر.

حد باز

قرارداد k را محدود نمی‌کند و بازهی عقب‌افتاده‌ی طولانی با k زیاد هزینه‌ی تقریباً خطی دارد ($k=399$ حدود ۱/۱۶ میلیون گس). تطبیق با gasLimit تولیدی هنوز انجام نشده است.

واجدان رأی: وضعیت در لحظه‌ی ساخت پیشنهاد

ثبت در Registry



statusNonce با هر تغییر وضعیت یک واحد زیاد می‌شود. activeCheckpoints [آدرس] آرایه‌ی فقط‌افزودنی {nonce, active} است؛ wasActiveAt(addr, nonce) با جست‌وجوی دودویی جواب می‌دهد. هر مؤسس با {true} شروع می‌کند.

مشکل و اصل راه‌حل



رأی‌دهنده‌ی یک پیشنهاد باید هنگام ساخت همان پیشنهاد واجد بوده باشد؛ وگرنه کسی می‌تواند بعد از ساخت پیشنهاد وارد شود و ترکیب مجلس را عوض کند. وضعیت در لحظه‌ی ساخت پیشنهاد ثبت می‌شود و رأی با آن سنجیده می‌شود.

بنیاد و هیأت



FoundationDAO: membershipNonce و memberSinceNonce (عضوی که پس از ساخت پیشنهاد افزوده شده رأی نمی‌دهد). هیأت: boardVersion در رأی و اجرا چک می‌شود.

قاعده‌ی رأی



پیشنهاد‌های Registry، Treasury و Distributor شمارنده را در createdAtNonce نگه می‌دارند. رأی معتبر یعنی wasActiveAt(voter, createdAtNonce) و همین حالا هم isValidator(voter) درست باشد.

شماره‌ی slot برای بررسی‌های genesis

Registry: everActivated در statusNonce، ۲۰ slot در ۳۶ و activeCheckpoints در ۳۷
FoundationDAO: membershipNonce در ۵ slot و memberSinceNonce در ۶
Distributor: تاریخچه و پیشنهاد‌های نرخ در slot‌های ۲۴ تا ۲۸ (همه صفر)

بلاک‌چین سور – ارائه به تیم فنی

FoundationDAO: حکمرانی داخلی بنیاد

تابع	نصاب	کار
<code>proposeAddMember(description, name, account)</code> <code>proposeRemoveMember</code>	دوسوم (۱۰ از ۱۵)	افزودن یا حذف عضو
<code>proposeSendETH(description, to, amount)</code>	اکثریت ساده (۸)	انتقال سورن از موجودی بنیاد
<code>proposeSendERC20(description, token, to, amount)</code>	اکثریت ساده (۸)	انتقال توکن
<code>proposeExecute(description, target, value, data)</code>	اکثریت ساده (۸)	فراخوانی دلخواه؛ value باید صفر باشد و انتقال سورن فقط با SendETH ممکن است
<code>getProposal</code> ، <code>getProposalMeta</code> ، <code>requiredVotesNow</code> ، <code>vote(id)</code> <code>hasVoted</code>	عضو / view	رای پیشنهاددهنده خودکار شمرده می‌شود

ایزوله

هیچ قرارداد دیگری را نمی‌شناسد و به خزانه‌ی ولیدیتورها وصل نیست.

موجودی

موجودی اولیه‌ی ۲۰ میلیون سورن در alloc نوشته می‌شود.

اعضا

اعضای اولیه (۱۵ نفر) در genesis seed می‌شوند؛ عضو افزوده‌شده بعد از ساخت یک پیشنهاد در آن رای نمی‌دهد.

IdentityRegistry: هویت کاربران

تابع	چه کسی	کار
<code>registerIdentity(personType, name)</code>	هر آدرس	خوداظهاری: شخص حقیقی یا حقوقی و نام
<code>setPhoneVerified</code> ، <code>setTelegramVerified</code>	identityOracle	نتیجه‌ی وریفای موبایل و تلگرام
<code>setKycVerified(addr, verified, commitment)</code>	identityOracle	KYC اختیاری؛ فقط commitment هش شده با salt
<code>migrateIdentity(oldAddr, newAddr)</code>	identityOracle	انتقال هویت پس از تغییر آدرس
<code>hasIdentity</code> ، <code>getVerificationStatus</code>	view	خواندن وضعیت هویت
<code>setIdentityOracle(newOracle)</code>	فقط FoundationDAO	چرخش کلید identityOracle

تطبیق آف‌چین

کوئری «آیا این مشخصات با آدرسی ثبت شده» یک API آف‌چین است، نه تراکنش.

پیش‌نیاز رأی

ثبت هویت (`hasIdentity`) برای رأی‌دادن در انتخاب هیأت لازم است.

حریم خصوصی

شماره‌ی موبایل، آیدی تلگرام و مدارک KYC هرگز روی زنجیره نمی‌روند؛ فقط وضعیت وریفای و commitment.

SurenSale: فروش ۶ ماهه و پنجره‌ی گذار

reportPayment(buyer, tomanAmount, paymentReference)



فقط paymentOracle؛ idempotent با [paymentReference] processedPayments. اگر کلید اوراکل لو برود، مهاجم فقط می‌تواند پرداخت ساختگی گزارش دهد، نه قیمت را عوض کند. موجودی را دوره‌ای و تکه‌تکه واریز کنید تا سقف زیان محدود بماند.

block.timestamp قیمت از



قیمت هر سورن فقط از زمان بلاک محاسبه می‌شود: ماه ۱ تا ۶ = ۱۰۰، ۱۰۳، ۱۰۶، ۱۰۹، ۱۱۳ و ۱۱۶ تومان (جدول ثابت)؛ سورن‌ها از همان موجودی قرارداد منتقل می‌شوند. saleStartTime تنها immutable است و قرارداد در genesis نیست.

()advanceTransitionPrice



بدون مجوز؛ قیمت روزهای کامل‌شده را به‌ترتیب اعمال می‌کند (حداکثر ۴۵ تکرار در کل عمر). reportPayment در دوره‌ی گذار قبل از قیمت‌گذاری خودش آن را صدا می‌زند؛ پس قیمت همیشه به «امروز» رسیده است.

پنجره‌ی گذار ۴۵ روزه



پس از ۱۸۰ روز، قیمت روزانه $\pm 0.5\%$ می‌شود: بالاتر از ۱۲۰٪ میانگین متحرک ۷ روزه‌ی حجم فروش افزایش، پایین‌تر از ۸۰٪ کاهش؛ کف ۹۰ و سقف ۱۵۰ تومان؛ دقت میلی‌تومان.

setPaymentOracle و sweepUnsold(to) فقط FoundationDAO (از مسیر proposeExecute)؛ sweepUnsold پس از پایان فروش و گذار. timeRemaining، getBalance و transitionTimeRemaining فقط خواندنی‌اند.

چهار کلید عملیاتی مستقل

کلید	قرارداد میزبان	کار	چرخش توسط
distributionOracle	BlockRewardDistributor	فراخوانی distributeRewards در هر epoch	هیأت ولیدیتورها (۳ رأی)
verifier	ValidatorsRegistry	ثبت تصمیم‌های وضعیت ولیدیتور (فعال‌سازی، تعلیق، بازگشت، تخلف پیش از خروج)	هیأت ولیدیتورها (۳ رأی)
identityOracle	IdentityRegistry	ثبت وریفای موبایل، تلگرام و KYC؛ انتقال هویت	فقط FoundationDAO
paymentOracle	SurenSale	گزارش پرداخت‌های تومانی تأییدشده	فقط FoundationDAO (اکثریت ساده)

جداسازی

هر کلید فقط یک کار می‌کند و هیچ دو سرویسی کلید مشترک ندارند. دو کلید مربوط به اجماع زیر نظر هیأت ولیدیتورها و دو کلید عملیات بنیاد زیر نظر خود بنیاد است.

نگهداری

همه‌ی کلیدها در HashiCorp Vault با policy و token جدا و کمترین دسترسی؛ نه فایل خام و نه متغیر محیطی. حساب هر سرویس باید سورن کافی برای گس داشته باشد.

چرخش

کلید جدید با تراکنش حکمرانی تعیین می‌شود؛ سرویس پیش از هر چرخه کلید فعلی قرارداد را می‌خواند و اگر کلیدش دیگر معتبر نیست متوقف می‌شود و هشدار می‌دهد.

RewardRouter: چرا لازم است و هر چرخه چه می‌کند

ریوارد بلاک یک اعتبار مستقیم روی state است، بدون فراخوانی EVM و بدون رویداد؛ پس قرارداد خودش نمی‌فهمد چه دریافت کرده یا به چه کسی بدهد. RewardRouter یک daemon کاملاً خودکار (Node.js، TypeScript، viem، SQLite، pino) است که هر ۲۳ ساعت و ۱۰ دقیقه بازه‌ای از بلاک‌ها را از خود زنجیره محاسبه و با یک تراکنش تسویه می‌کند.

۱	پیش‌شرط‌ها	کلید همین سرویس هنوز distributionOracle است (getDistributionOracle)؛ timeUntilNextDistribution. در غیر این صورت توقف و هشدار.
۲	تعیین بازه	$S = \text{lastSettledBlock}$ و $L = \text{eth_blockNumber}$ (آخرین بلاک مهرشده)؛ بازه $[S+1, L]$. اگر $L < S+1$ کاری نیست. QBFT نهایی‌شدن فوری دارد، پس margin اضافه لازم نیست.
۳	اسکن هر بلاک	miner از eth_getBlockByNumber؛ ریوارد از trace_block؛ کارمزد از رسید تراکنش‌ها؛ صلاحیت تولیدکننده در ارتفاع $N-1$.
۴	تجمیع	تعداد بلاک و جمع ریوارد و کارمزد هر miner؛ آرایه‌ها بر اساس مقدار عددی آدرس صعودی و بدون تکرار.
۵	کنترل پیش از ارسال	معادله‌ی موجودی، مقایسه با سقف پاداش، $\text{sum}(\text{blocksMined}) \geq$ اندازه‌ی بازه.
۶	ارسال و تطبیق	ثبت پایدار، امضا، broadcast؛ سپس تطبیق رسید و lastSettledBlock (اسلاید ۲۵).

RewardRouter: محاسبه‌ی مبالغ و کنترل‌های پیش از ارسال

معادله‌ی موجودی



برای بازه‌ی $[f, t]$: $B(t) - B(f-1) = R + F + M + X - P$:
 عضویت ورودی، X واریز دیگر، P خروجی توزیع‌های بازه). اگر نیست یا $F \neq \text{totalFees}$ ،
 ارسال نمی‌شود و هشدار می‌دهد.

ریوارد و کارمزد



ریوارد: فقط entry از نوع reward در `trace_block`. کارمزد هر تراکنش: $\text{gasUsed} \times (\text{effectiveGasPrice} - \text{baseFeePerGas})$ با `baseFeePerGas` از هدر همان بلاک (اگر وجود ندارد یا صفر است: $\text{gasUsed} \times \text{effectiveGasPrice}$). کارمزد هرگز از `trace_block` حساب نمی‌شود.

صلاحیت تولیدکننده



ولیدیتوری که هنگام تولید `Active` بوده، حتی بعد از خروج یا تعلیق پاداش بلاک‌هایش را می‌گیرد: `isValidator` روی ارتفاع $N-1$ (نود آرشیوی) یا بازسازی از `StatusDecisionRecorded` و `ExitRequested` با مؤسسان `Active` از بلاک صفر. شمارش ناقص یا صلاحیت نامعلوم یعنی ارسال نشدن همان بازه.

سقف پاداش



`maxRewardsForRange(from, to)` از قرارداد خوانده می‌شود. اگر پاداش واقعی بیشتر باشد، کل بازه متوقف می‌شود و `totalRewards` هرگز کم نمی‌شود؛ `lastSettledBlock` نمی‌گذرد. اگر کمتر باشد، ناهماهنگی نرخ ثبت و هشدار داده می‌شود.

RewardRouter: ارسال ایمن، crash و جدول‌ها

جدول	ستون‌های کلیدی
epochs	بازه، مبالغ، confirmed_tx_hash، nonce، status، calldata
epoch_txs	هر تلاش ارسال: tx_hash، raw_signed_tx، fee_params، receipt_status
validator_epoch_shares	epoch_id، آدرس ولیدیتور، blocks_mined
scan_progress	last_scanned_block؛ فقط بهینه‌ی خواندن

وضعیت epoch	معنا
prepared	بازه، مبالغ، calldata و nonce رزرو شد؛ هنوز ارسال نشده
submitted	broadcast شد؛ رسید نیامده
unknown	timeout یا قطع ارتباط؛ هرگز شکست قطعی نیست
confirmed / reverted	رسید با status ۱ یا ۰؛ هر دو نهایی‌اند

ترتیب اجباری ارسال و بازیابی

- تراکنش را با nonce رزرو شده امضا کن؛ ۲. tx_hash و raw_signed_tx را با commit پایدار در epoch_txs بنویس؛ ۳. فقط پس از آن eth_sendRawTransaction بزن. بدون ردیف پایدار هرگز broadcast نکن.
- پس از restart: رسید همه‌ی hash‌های ثبت‌شده، lastSettledBlock و nonce تأییدشده را بخوان. بدون رسید و nonce هنوز مصرف‌نشده: همان بایت‌ها را بازپخش کن. ارسال با nonce دیگر ممنوع است؛ فقط جایگزینی با همان nonce و همان calldata و گس‌پرایس بالاتر ($\leq 10\%$).
- اگر nonce مصرف شده و lastSettledBlock = toBlock، یکی از نسخه‌های جایگزین کار کرده (confirmed). حالت ناسازگار دیگر یعنی توقف و هشدار. پارامترهای جایگزینی (REPLACE_AFTER، MAX_REPLACEMENTS) پیشنهادی‌اند و باید روی txpool واقعی آزموده شوند.

Verifier: وظایف، ورودی‌ها و معماری

ورودی‌ها



admin_peers از یک نود Besu اختصاصی که static peer همه‌ی ولیدیتورهاست (remoteAddress و nodeID)؛ IP-geolocation روی remoteAddress؛ از زنجیره: miners، getValidatorInfo، getValidators، بلاک‌های اخیر و ارتفاع فعلی؛ اپ همراه: آدرس، شماره، آیدی تلگرام، nodeID (بخش کلید عمومی enode).

وظایف



احراز هویت (موبایل و تلگرام)، سنجش سلامت نود ولیدیتورها، نگهداری داده‌ی حساس آف‌چین (شماره، آیدی، nodeID) که هرگز روی زنجیره نمی‌رود، و احراز مداوم اینکه نود در ایران است.

خروجی‌ها



تراکنش فقط هنگام تغییر واقعی وضعیت و همراه evidenceHash = keccak256 (بسته‌ی شواهد)؛ اعلان به ولیدیتور؛ لاگ کامل هر چک و تصمیم. هر هفت تابع کاری require صریح دارند و در صورت برقرارنبودن شرط revert می‌کنند، نه اینکه بی‌اثر موفق شوند.

معماری



جداسازی «چک‌کردن» از «امضاکردن»: چند نمونه‌ی polling جانشین برای scheduler؛ Node.js، TypeScript، viem، better-sqlite3 (نه API عمومی)؛ جداسازی «چک‌کردن» از «امضاکردن»: چند نمونه‌ی polling جانشین برای دسترس‌پذیری، یک مرجع تصمیم و یک مرجع امضا؛ نمونه‌ی دوم هم‌زمان فقط revert می‌شود چون قرارداد شرط وضعیت را چک می‌کند.

Verifier: قواعد تصمیم هر بررسی ساعتی

هر بررسی برای هر ولیدیتور Active پنجره‌ی ۴ ساعته‌ی منتهی به «اکنون» را بر پایه‌ی زمان بلاک‌ها ارزیابی می‌کند و دقیقاً یکی از سه نتیجه می‌دهد: OK، INACTIVE_CONFIRMED یا UNKNOWN(کد). اولین قاعده‌ی برقرار تعیین‌کننده است.

ترتیب	نتیجه	شرط
۱	UNKNOWN(MONITOR_UNHEALTHY)	نود پایش همگام نیست، RPC لازم شکست خورده، یا داده‌ی پنجره ناقص است
۲	UNKNOWN(OBSERVATION_GAP)	فاصله‌ی دو بررسی متوالی از حد مجاز بیشتر یا سرویس در پنجره قطعی داشته
۳	UNKNOWN(EMERGENCY_WINDOW)	پنجره با بازه‌ی بازیابی اضطراری هم‌پوشان است
۴	OK	ولیدیتور در پنجره دست‌کم یک بلاک پیشنهاد داده است
۵	UNKNOWN(NETWORK_UNHEALTHY)	فاصله‌ی دو بلاک بیش از ۱۲۰ ثانیه یا سهم ارتفاع‌های با دور بیش از صفر بالای ۲۰٪
۶	UNKNOWN(GENERAL_OUTAGE)	دست‌کم ۲۰٪ ولیدیتورهای Active هم‌زمان ساکت‌اند؛ هیچ‌کدام تخلف فردی نیست
۷	UNKNOWN(NO_OPPORTUNITY)	نوبت مورد انتظار ولیدیتور در پنجره کمتر از ۳ بوده
۸	INACTIVE_CONFIRMED	هیچ بلاکی، دست‌کم ۳ نوبت مورد انتظار همگی از دست رفته، و قواعد ۱ تا ۷ برقرار نیست

Verifier: آستانه‌ها، فعال‌سازی و کارهای بدون مجوز

فعال‌سازی و بازگشت



فعال‌سازی پس از ۱ هفته probation، موفقیت $\leq 95\%$ و فقط اگر نود واقعی روشن و همگام باشد. بازگشت پس از ۴۸ ساعت و $\leq 95\%$. عددهای آستانه‌ی قواعد (۱۲۰ ثانیه، ۲۰٪، ۳ نوبت) پیشنهادی‌اند و پیش از راه‌اندازی باید نهایی شوند.

دوره و آستانه‌ها



بررسی هر ۶۰ دقیقه؛ هشدار اولیه ۱۵ دقیقه، هشدار جدی ۱ ساعت، تعلیق پس از ۴ ساعت غیرفعالی تأییدشده. هر آستانه در اولین بررسی پس از عبور دیده می‌شود؛ پس تعلیق ۴ تا ۵ ساعت پس از آخرین بلاک ثبت می‌شود.

خرابی خود سرویس



حفره‌ی مشاهده‌ی خود سرویس هرگز غیرفعالی ولیدیتور حساب نمی‌شود؛ شمارش ۴ ساعته از داده‌ی کامل پس از رفع خرابی دوباره شروع می‌شود. غیرفعالی پس از درخواست خروج هم تخلف نیست؛ فقط تا ۷۲ ساعت رفتار پیش از درخواست بررسی می‌شود.

کارهای بدون مجوز



برای معطل‌نماندن پرونده‌ها سرویس این‌ها را صدا می‌زند:
`resolveMassFailureCheck`، `assertDeliveryDisputed`،
`executeUncontestedSlash`، `resolveAppealIfExpired`،
`resolveDeliveryDisputeIfExpired`، `syncBoard`، `fillVacancies`
`.refreshBoard`

PaymentReporter و Identity Service

PaymentReporter (کلید paymentOracle)

واسط درگاه پرداخت و webhook SurenSale: درگاه را دریافت می‌کند، پرداخت را مستقل دوباره با درگاه راستی‌آزمایی می‌کند و سپس reportPayment را با paymentReference یکتا می‌فرستد.

تکرار webhook بی‌خطر است چون قرارداد پرداخت تکراری را رد می‌کند.

دو اندپوینت برای اپ خرید: POST /api/v1/purchase/initiate (شروع خرید، ساخت paymentReference و آدرس درگاه) و یک GET برای پیگیری وضعیت؛ یک مکانیزم رزرو موجودی از فروش مضاعف جلوگیری می‌کند.

هر خطا یا اختلاف بین درگاه و زنجیره یعنی هشدار فوری و پیجینگ؛ هیچ پرداختی بدون راستی‌آزمایی مستقل گزارش نمی‌شود.

Identity Service (کلید identityOracle)

وریفای کد پیامکی و تلگرامی؛ نتیجه با setPhoneVerified و setTelegramVerified ثبت می‌شود. داده‌ی خام (شماره و آیدی) هرگز روی زنجیره نمی‌رود.

KYC اختیاری از طریق ارائه‌دهنده‌ی eKYC؛ فقط commitment هاش شده با salt (setKycVerified).

کوئری «تطبیق» (آیا این مشخصات با یک آدرس ثبت‌شده می‌خواند) همیشه API آف‌چین است، نه تراکنش، چون خود ورودی کوئری حساس است.

ثبت‌نام در IdentityRegistry با خود کاربر است؛ سرویس فقط وضعیت‌های وریفای را می‌نویسد.

زیرساخت مشترک: Vault، Docker Compose، نودها و اکسپلورر

Docker Compose



هر سرویس یک container؛ SQLite روی volume پایدار؛ لاگ ساخت‌یافته با pino؛ هشدار پیچینگ با چرخه‌ی on-call مشترک؛ هشدار اگر ۲۴ ساعت گذشت و epoch جدید confirmed نشد.

HashiCorp Vault



هر نرم‌افزاری که کلید خصوصی دارد از Vault استفاده می‌کند؛ به‌ازای هر سرویس token و policy مستقل با کمترین دسترسی. زیرساخت امضای ریموت برای جداسازی امضا از polling.

Blockscout



اکسپلورر عمومی. وریفای سورس قراردادهای فقط وقتی ممکن است که با solc 0.8.24 بدون vialR و بدون constructor برای قراردادهای genesis کامپایل شده باشند.

نودها



یک نود Besu با API های TRACE و ADMIN و RPC عمومی برای سرویس‌ها؛ یک نود پایش به‌عنوان static peer همه‌ی ولیدیتورها. از static-nodes.json استفاده کنید؛ -- bootnodes روی Besu ویندوز با هر enode خطای «<:> Illegal char» می‌دهد.

بازیابی اضطراری اجماع: گذار blockheader

۱ تشخیص

قفل شدن زنجیره (نصاب QBFT برآورده نمی‌شود) را از توقف تولید بلاک تشخیص دهید؛ شمارش نودهای روشن در برابر مجموعه‌ی فعال. بعضی ولیدیتورها ممکن است بدون نود واقعی فعال شده باشند.

۲ احیای نودها

همه‌ی نودهای صادق را هم‌زمان ری‌استارت کنید؛ ری‌استارت جزئی معمولاً کافی نیست. نودهای مفقود را راه بیندازید و ببینید نصاب بازمی‌گردد یا نه.

۳ گذار به blockheader

اگر هنوز قفل است، در یک ارتفاع آینده transitions.qbft موقتاً انتخاب ولیدیتور را به حالت blockheader با فهرست صریح می‌برد؛ این تغییر پیکربندی را باید همه‌ی نودها پیش از آن ارتفاع اعمال کنند.

۴ بازگشت به قرارداد

پس از بازیابی، گذار دوم دوباره حالت contract با آدرس ValidatorsRegistry را برمی‌گرداند. فهرست صریح باید با وضعیت قرارداد هم‌خوان باشد.

۵ وضعیت آزمون و فرایند

روش فنی (احیای نود و گذار رفت‌وبرگشت) روی Besu واقعی آزموده شده است؛ فرایند سازمانی تأیید ۵ از ۷ مسئول مستقل و ابزار کمکی هنوز نوشته نشده است.

امنیت و مرز اعتماد

قرارداد چه چیزی را تأیید نمی‌کند



صحت totalRewards و totalFees و انتساب بلاک‌ها به ولیدیتورها را از distributionOracle می‌پذیرد و با زنجیره تطبیق نمی‌دهد. محافظ‌ها: سقف پاداش، everActivated، موجودی قرارداد و چرخش کلید توسط هیأت.

قرارداد چه تضمین می‌کند



بازهی تسویه بدون تکرار، هم‌پوشانی و فاصله؛ سقف پاداش هر بازه؛ صعودی بودن و یکتایی آدرس‌ها؛ نصاب و تأخیر هر حکمرانی؛ پرداخت فقط به آدرس‌هایی که حداقل یک‌بار مشروع Active بوده‌اند؛ انتقال سورن با call و بررسی نتیجه و nonReentrant روی مسیرهای حساس.

ایزوله‌سازی



FoundationDAO هیچ مسیری به خزانه‌ی ولیدیتورها ندارد؛ خرج Treasury فقط با هیأت و زیر دو سقف؛ هیچ ادمین مرکزی و هیچ مسیر دورزدن اضطراری سقف پاداش وجود ندارد.

سطح حمله‌ی کلیدها



distributionOracle و verifier نقاط تمرکز اعتماداند. مهاجم با کلید paymentOracle فقط پرداخت ساختگی تا سقف موجودی SurenSale می‌تواند گزارش دهد. برای تک‌Verifier معافیت جمعی فقط جریمه‌ی مالی را متوقف می‌کند و خطای او روی مجموعه‌ی اجماع اثر دارد.

دوره‌های آزمون Besu و وضعیت هر دور

چهار دور آزمون مستند شده‌اند؛ دور جاری و معتبر برای کد فعلی دور چهارم (v4) است

دور	تاریخ	کد	دامنه و وضعیت
دور اول	۲۷ تا ۲۸ سپتامبر ۲۰۲۶	نسخه‌ی پیشین؛ Besu ۲۶/۹/۰	ساخت شبکه‌ی ۴نودی از صفر طبق runbook؛ سناریوهای A تا D (عضویت، تعلیق، جریمه، هیأت و خزانه) با تراکنش واقعی؛ دو یافته‌ی کامپایل و راه‌اندازی. نتایج آن برای کد فعلی معتبر نیست؛ فقط دانش عملیاتی نگه داشته شده است
دور دوم (v2)	۲۸ تا ۲۹ سپتامبر ۲۰۲۶	کد پیش از سیاست everActivated	ساخت شبکه از صفر، چرخه‌ی کامل Registry، هیأت، خزانه، FoundationDAO، IdentityRegistry و هر دو مرحله‌ی بازیابی اضطراری. اجرا شد؛ فقط نتیجه‌ی فاز توزیع پاداش آن دیگر معتبر نیست، چون پاداش هنوز با isValidator پرداخت می‌شد
دور سوم (v3)	۳۰ سپتامبر ۲۰۲۶	کد با everActivated؛ شبکه‌ی ۴نودی Besu ۲۶/۹/۰	پنج فاز با تراکنش واقعی، همه موفق؛ تأیید سیاست everActivated و اولین توزیع واقعی. پیش از اصلاحات L01 تا L07 اجرا شد، پس برای قراردادهایی که آن اصلاحات تغییرشان داده‌اند دیگر معتبر نیست
دور چهارم (v4)	۲ تا ۴ اکتبر ۲۰۲۶	کد فعلی؛ Besu ۲۶/۹/۰	۹۷ ردیف در چهار گذر پیاپی روی شبکه‌های Besu مستقل؛ جزئیات در پنج اسلاید دور چهارم (اسلایدهای ۳۷ تا ۴۱). تنها دوری که روی نسخه‌ی فعلی کد اجرا شده است

این بخش آمادگی production را اعلام نمی‌کند.

دور اول (۲۷ تا ۲۸ سپتامبر): راه‌اندازی شبکه و یافته‌ها

۲۶/۹/۰ Besu روی ویندوز؛ شبکهٔ ۴نودی که بعداً ۵نودی شد؛ طبق runbook مالک؛ حدود ۲ ساعت؛ روی نسخه‌ی پیشین کد

موضوع	نتیجه
پارامترهای آزمایشی	۳۰۰ probation ثانیه، ۱۲۰، DELIVERY_DISPUTE_GRACE_PERIOD ۱۸۰، APPEAL_VOTING_PERIOD ۱۲۰، APPEAL_FILING_WINDOW ۶۰، MASS_DEMOTION_WINDOW ۱۲۰، recovery ۱۲۰، exitCooldown؛ سقف‌های خزانه ۱۰۰۰ و ۵۰۰۰ سورن؛ verifier یک آدرس آزمایشی؛ ولیدیتورهای seed برابر ۴ نود واقعی
کامپایل	با ۰/۸/۲۴ solc و بدون vialR، خطای Stack too deep در distributeRewards دیده شد. در نسخهٔ آزمایشی با استخراج prepareEpoch رفع شد؛ بدون تغییر در require، رویداد یا ترتیب اجرا
hard-forkها	نبود بلاک فعال‌سازی fork در config باعث (SHR) Invalid opcode 0x1c در getValidators شد؛ با homestead تا london در بلاک ۰ و zeroBaseFee رفع شد
--bootnodes روی Besu ویندوز	با هر enode خطای Illegal char <: > می‌دهد (حتی با رشتهٔ ساختگی)؛ راه‌حل static-nodes.json در پوشهٔ داده‌ی هر نود
genesis و storage	storage با استقرار واقعی هر قرارداد روی Hardhat و محاسبهٔ slotهای mapping و array استخراج شد؛ round-trip روی Registry پیش از Besu برابر بود. روی هر چهار نود، genesis hash، timestamp و eth_getCode(0x3333...3333) یکسان بود (۳۷,۲۱۸ نویسه، ...5e39eb51ab25 sha256)
انتخاب پیشنهاددهنده	با qbft.validatorcontractaddress، بلاک‌های ۱ تا ۸ دقیقاً round-robin بین چهار ولیدیتور seed چرخید؛ یعنی چرخش از قرارداد خوانده می‌شود
انحرافات از runbook	سازنده‌ی helperهای GenesisSeed بدون آرگومان بود؛ آرایهٔ Registry از ۷ به ۴ تقلیل یافت؛ verifier پیش‌فرض عوض شد؛ Board و FoundationDAO با state خالی استقرار یافتند، چون هیچ سناریویی به آن‌ها نیاز نداشت

این دور روی نسخه‌ی پیشین کد بود؛ فقط دانش عملیاتی آن (fork، bootnodes، timestamp) هنوز راهنماست.

دور اول: سناریوهای A تا D روی Besu

موضوع	نتیجه
A: عضویت و فعال‌سازی	requestMembership با ۵۲۰,۰۰۰ سورن (محاسبه زنده)، انتظار واقعی ۳۰۰ ثانیه، recordActivation؛ isValidator و getValidators فوراً به‌روز شدند. نامزد پنجم که فقط یک EOA بود هرگز بلاک نداشت؛ هر پنج نوبت یک شکاف ثابت ۱۳ ثانیه‌ای (بلاک ۳ ثانیه + timeout دور ۱۰ ثانیه) دیده شد: رفتار درست تحمل‌خطای BFT نه عیب. پس از راه‌اندازی نود واقعی با همان کلید، بلاک ۱۷۳ را خودش ساخت و شکاف‌ها رفتند
B: تعلیق و تطبیق نصاب	recordSuspension در بلاک ۱۸۸ (۲ decisionId)؛ getValidators در فراخوانی بعدی ولیدیتور را حذف کرد. ۱۵ بلاک بعد با فاصله ۳ ثانیه و بدون شکاف، و ولیدیتور تعلیق‌شده دیگر پیشنهاد نداد. سؤال «زیر نصاب زنجیره می‌ایستد؟» آزموده نشد (۵ به ۴، بالاتر از نصاب)
C: جریمه بی‌اعتراض	resolveMassFailureCheck معاف نکرد: ۱ از ۵ دقیقاً ۲۰٪ است و قرارداد «بیشتر از ۲۰٪» می‌خواهد (ترتیب A پیش از B/C مهم بود). confirmDelivery، انتظار ۱۳۰ ثانیه از پنجره اعتراض بدون fileAppeal و سپس executeUncontestedSlash موفق شد. چون وثیقه مؤسس صفر است، جریمه ۱٪ برابر ۰ wei بود؛ سازوکار تأیید شد، اثر مالی نه
D: هیأت و خزانه	خزانه با ۲۰۰۰ سورن پر شد؛ چهار ولیدیتور فعال (voteFor(self)، registerIdentity و refreshBoard زدند و عضو هیأت شدند. proposeApproveBudget برای ۱۰۰ سورن ۱ actionId گرفت. رأی پیشنهاددهنده خودکار شمرده می‌شود (رأی چهارم صریح با already voted رد شد؛ اشتباه اسکرپت من بود نه قرارداد). با ۳ رأی، موجودی گیرنده از ۰ به دقیقاً ۱۰۰ سورن رسید

اثر مالی جریمه روی عضو پرداخت‌کننده و رفتار زیر نصاب در این دور آزموده نشد.

یافته‌های دوره‌های دوم و سوم (۲۸ تا ۳۰ سپتامبر)

دور دوم: شبکه ۱۵ نودی و هر ۱۰ فاز. دور سوم: شبکه ۴ نودی با ۲۶/۹/۰ Besu و کد دارای سیاست everActivated. هر دو روی نسخه‌های پیشین کد

یافته	نتیجه / پیامد عملی
دور دوم: چرخه قراردادها	چرخه کامل Registry (عضویت، تعلیق، معافیت جمعی، اعتراض، اختلاف تحویل، بازگشت، خروج، تغییر پارامتر)، بازطراحی refreshBoard، خزانه، IdentityRegistry، FoundationDAO و هر دو مرحله بازیابی اضطراری (N04) با تراکنش واقعی تأیید شد
دور دوم: توزیع پاداش	اولین distributeRewards واقعی روی کد قدیمی (isValidator): از ۱۵ پیشنهاددهنده فقط ۴ نفری که هنوز فعال بودند پاداش گرفتند. همین رفتار بعداً مشکل شناخته شد و با everActivated عوض شد؛ نتیجه این فاز دیگر معتبر نیست
قفل زنجیره با ولیدیتور بدون نود	وقتی نصاب از شمار نودهای روشن بیشتر شد زنجیره حدود ۵۴ دقیقه ایستاد (یافته بحرانی عملیاتی دور دوم)
دور سوم: سیاست everActivated	پنج فاز با تراکنش واقعی، همه موفق: مؤسسان genesis از بلاک صفر everActivated=true می‌گیرند و اولین توزیع واقعی موفق می‌شود؛ پرچم ماندگار است و آدرس ساختگی رد می‌شود
سرعت واقعی تولید بلاک	حدود ۲/۹۵ ثانیه برای هر بلاک، کمی سریع‌تر از ۳ ثانیه‌ی اسمی؛ چک سقف فیزیکی آن زمان دو بار به آن خورد و بعداً حذف شد (L03)
genesis با timestamp در آینده	زنجیره تا فرارسیدن آن زمان بلاک نمی‌سازد و بدون خطا در لاگ شبیه قفل اجماع دیده می‌شود (حدود ۳۰ دقیقه جلوتر، حدود ۱۵ دقیقه معطلی)؛ timestamp را با فاصله کم از «اکنون» بگذارید

نتایج نسخه‌ی فعلی کد در اسلایدهای دور چهارم (v4) آمده است.

دور چهارم (v4): دامنه، گذرها و شمار نتایج

موضوع	نتیجه
محیط	۲۴/۸/۰، solc ۲۶/۹/۰، Besu با optimizer و بدون vialR؛ یک میزبان ویندوز با ۵ تا ۶ فرایند نود برای هر شبکه؛ قراردادهای بایت‌به‌بایت برابر Plan (هش ۱۳ فایل انگلیسی و ۱۳ فایل فارسی)
چهار گذر	گذر اول (۲ اکتبر): L01، L02، L04، L05، L07 و رگرسیون everActivated. گذر تکمیلی (۳ اکتبر): تکرار L05 با پیش‌شرط درست، حسابداری مستقل D02، گروه‌های E و F. گذر دوم: آزمون ترکیبی Net-F6، اسکن A14، تکرار F07. گذر سوم: C-L04-6 با seed درست و پوشش گروه A روی شبکه‌های تکمیلی. گذر چهارم (۴ اکتبر): رفع A09 و C-L04-6b
شمار ردیف‌ها	۹۷ ردیف: ۹۰ NOT-RUN، ۱ SUPERSEDED، ۶ FAIL، ۰ PASS؛ شمارش با اسکریپت از خود جدول است
FAIL	هیچ ردیف FAIL باقی نمانده. A09-spec (گذر سوم) FAIL بود: lastBoardRefreshAt در genesis آزمایشی صفر بود، یعنی شکاف ابزار آزمایشی نه نقص قرارداد؛ در گذر چهارم اصلاح و با کنترل منفی تأیید شد و ردیفش SUPERSEDED است
۶ ردیف SUPERSEDED	چهار ردیف اولیه C-L05 (با پیش‌شرط نادرست)، ردیف A09-spec و اجرای اول C-L04-6b (انتظار اشتباه در اسکریپت)؛ همگی با اجرای دوباره جایگزین شدند و برای شفافیت نگه داشته شده‌اند
۵ نوع شاهد (جدا از هم)	baseline (قراردادهای دست‌نخورده)؛ test-fork (سه ثابت زمانی فشرده، شاهد baseline نیست)؛ شبیه‌سازی با eth_call (چیزی استخراج نمی‌شود)؛ TEST-ONLY-SEED (state نوشته شده مستقیم در genesis)؛ benchmark مصنوعی (گروه E)
توقف‌های بحرانی	همه پاک: هیچ ناهمخوانی هش، هیچ توزیع بالاتر از سقف، هیچ پرداخت به آدرس هرگز-فعال نشده و هیچ رأی نامعتبر پذیرفته شده‌ای دیده نشد

همه‌ی اجراها روی یک میزبان بودند و این جدول آمادگی production را اعلام نمی‌کند.

دور چهارم: نتایج تأییدشده – شبکه، ولیدیتورها و حکمرانی

موضوع	نتیجه
storage و genesis	توزیع شش قرارداد با code و storage مستقیم در alloc کار کرد و هش eth_getCode با کامپایل برابر بود. A14: در ۲۰ genesis هیچ حساب یا slot ناخواسته‌ای نبود (کنترل منفی هر ۴ دست‌کاری را گرفت) و ریشه‌ی state بلاک ۰ در ۲۱ شبکه با alloc برابر بود
ولیدیتورها (گروه B)	تعليق، فعال‌سازی، بازگشت و تعليق هم‌زمان دو ولیدیتور با QBFT و ۱,۰۰۲ B06: validatorcontractaddress بررسی isValidator در برابر getValidators بدون اختلاف (~۵۱۲ بلاک اخیر). B07: miner(N) در Registry@(N-1) برای ۱۷ بلاک نمونه
اصلاحات L01 تا L07	قفل فاصله‌ی تغییر سهم در نقطه‌ی اجرا؛ اختیار زنده‌ی هیأت و نسخه‌ی ترکیب (C-L02، C-L05-3a/3b)؛ snapshot واجدان رأی (C-L04)؛ سقف و حکمرانی نرخ با نصاب منجمد و رد اجرای زود هنگام (C-L05-1 تا ۳)؛ ترتیب صعودی (C-L07)؛ سیاست (C-REG-2) everActivated
رأی عضو افزوده یا حذف‌شده	عضو افزوده‌شده پس از ساخت پیشنهاد روی آن رأی نمی‌دهد ولی روی پیشنهادهای بعدی مجاز است؛ عضو حذف‌شده‌شده روی پیشنهاد باز دوباره با «already voted» رد می‌شود بدون شمارش مضاعف؛ نصاب پیشنهاد باز در هر دو جهت منجمد است (C-L04-6 و 6b، هر کدام یک اجرا روی یک شبکه)
گروه A روی شبکه‌های تکمیلی	A01 تا A13 و A15 روی ۱۱ شبکه‌ی دوقلو با genesis یکسان برای ۱۵ شبکه‌ی تکمیلی (هم‌ارزی ۱۵ از ۱۵؛ A07 و A08 پاک). تنها شکست گذر سوم A09-spec بود؛ پس از اصلاح ابزار آزمایشی، ۴۳۸ ثانیه پس از genesis فراخوانی refreshBoard با پیام دقیق رد شد. مرز ۳۰ روز آزموده نشد

دور چهارم: تسویه، هماهنگی نرخ پاداش و هزینه‌ی گس

موضوع	نتیجه
تسویه (گروه D)	D01 تا D03 و D06 پاس. حسابداری مستقل برای D02 روی Net-D3 و برای آزمون ترکیبی Net-F6: عبور از transition در بلاک ۱۰۰ با بلاک‌های غیرخالی، کارمزد عادی، کارمزد عضویت و ورودی مستقیم؛ R دقیقاً برابر سقف؛ سوزاندن ۳۰٪ فقط روی کارمزد عادی. D04 و D05 فقط روی fork
transition پاداش (F01، F02)	قالب transitions.qbft سند پذیرفته شد؛ پاداش از بلاک X (شامل خودش) عوض می‌شود و تاریخچه‌ی قرارداد در همان بلاک، بدون اختلاف یک‌بلاکی
سقف پاداش (F03، F04)	پاداش واقعی بیشتر از سقف رد می‌شود و state تغییر نمی‌کند؛ پاداش کمتر از سقف می‌گذرد. F04: تراکنش استخراج‌شده روی Net-F3؛ روی Net-F6 فقط eth_call
ری‌استارت و نود ناهماهنگ (F05 تا F07)	تغییر فایل genesis و ری‌استارت یکی یکی نودها کافی است و زنجیره نایستاد؛ بدون ری‌استارت اثری ندارد. نود ناهماهنگ یا تازه‌واردی با genesis قدیمی در بلاک X رد می‌شود
هزینه‌ی گس (گروه E، benchmark)	≈ ۱۳۷,۶۰۰ گس به‌ازای هر payee و ≈ ۲,۸۱۰ گس به‌ازای هر تغییر نرخ داخل بازه. سنگین‌ترین ترکیب (payee, k=۳۹۹ ۱۵۰): تخمین ۲۲,۱۸۸,۲۳۹ و گس واقعی ۲۱,۹۹۲,۴۷۰. در ۱۵M گس شش ترکیب از ۲۵ ترکیب جا نشد. این اعداد gasLimit تولیدی را تعیین نمی‌کنند

D04، D05 و F04 روی Net-F6 به‌ترتیب شاهد fork و شبیه‌سازی‌اند، نه baseline.

دور چهارم: یافته‌های عملیاتی

موضوع	نتیجه
هماهنگی transition	genesis فقط هنگام راه‌اندازی خوانده می‌شود؛ نودی که transition را نداشته باشد در بلاک X با «World State Root does not match» رد می‌شود. با یک نود ناهماهنگ از ۵، چهار نود باقی‌مانده (دقیقاً نصاب) با تأخیر ≈ 10 ثانیه در نوبت آن نود ادامه دادند
اختلاف نرخ	قرارداد پاداش بیشتر از سقف را رد می‌کند ولی پاداش کمتر از نرخ مصوب را بی‌صدا می‌پذیرد؛ فقط ابزار بیرونی آن را می‌بیند
داده‌ی تاریخی برای Router	خارج از ≈ 512 بلاک اخیر (Bonsai) مقدار eth_getBalance برابر null و trace_block آرایه‌ی خالی است، بدون خطا؛ بازه‌ی ۲۳ ساعته $\approx 27,600$ بلاک است
موجودی گس اوراکل	هر distributeRewards در آزمون‌های واقعی ≈ 94 تا ≈ 136 SUR با gas price آزمایشی؛ در benchmark با 150 SUR $\approx 2,199$ payee
نود با genesis ناهماهنگ	بلاک نامعتبر را مدام تکرار می‌کند (۱,۱۱۱ بار در ≈ 36 دقیقه) و حافظه با heap پیش‌فرض ظرف ≈ 2 دقیقه از $\approx 6/0$ به $\approx 5/7$ گیگابایت رسید. OOM دیده نشد ولی نبود نشت اثبات نشد؛ نیازمند مداخله‌ی اپراتور
تراکنش با پیش‌پرداخت ناکافی	بیش از ۲۰ دقیقه بی‌صدا pending ماند (نه استخراج، نه رد)؛ ری‌استارت نودها pool را پاک کرد. یک بار دیده شد
ابزار مشاهده	debug_storageRangeAt و debug_accountRange روی نود Bonsai خالی برمی‌گردانند؛ A14 با مقایسه‌ی ریشه‌ی state انجام شد
رفتار قرارداد (نیازمند تصمیم)	بازگشت ولیدیتور به شکل مستندنشده به بسته‌شدن پرونده‌ی جریمه وابسته است؛ انتقال مستقیم به distributor هرگز توزیع نمی‌شود و در موجودی می‌ماند

گزینه‌های طراحی این یافته‌ها هنوز انتخاب نشده‌اند.

دور چهارم: اجرانشده‌ها، محدودیت‌ها و موارد باز

موضوع	نتیجه
اجرانشده (۱ مورد)	مسیر ترکیبی زمان واقعی L (مرحله‌ی L4 در C-L05-4): پیشنهاد، تأخیر واقعی ۷روزه، اجرا، $\leq 201,600$ بلاک، عبور از startBlock و تسویه با نرخ جدید. دو نیمه‌اش جدا شاهد دارند ولی هرگز با هم اجرا نشدند. برنامه‌ی اجرا (≈ 15 روز) فقط نوشته شده است
محدودیت ردیف‌های اجرانشده	F04 روی Net-F6 فقط شبیه‌سازی؛ D04 و D05 فقط fork؛ B07 فقط ۱۷ بلاک؛ A14 فقط genesis و ریشه‌ی state بلاک ۰؛ A09-fix فقط رد پیش از ۳۰ روز و ۳۳ شبکه‌ی قدیمی مقدار صفر را نگه می‌دارند؛ C-L04-6 و 6b هر کدام یک اجرا؛ F07 یک پنجره‌ی ≈ 36 دقیقه‌ای؛ همه تک‌میزبان
غیرقابل‌آزمون روی Besu	قفل ۱۸۰روزه‌ی تغییر سهم، انقضای ۳۰روزه‌ی پیشنهادها و ۳۰ refreshروزه‌ی هیأت
باز، و Besu آن‌ها را نمی‌بندد	مسیر رفع «پاداش واقعی < سقف» برای بازه‌ی گذشته؛ تصمیم gasLimit تولیدی؛ پیاده‌سازی و آزمون RewardRouter؛ صفحه‌ی حکمرانی نرخ در داشبورد؛ ابزار genesis تولیدی (T02) که باید lastBoardRefreshAt را برابر genesis timestamp بنویسد و assert کند؛ این الزام موجود پروژه است، نه تصمیم باز
تصمیم‌های انتخاب‌نشده	رویه‌ی اپراتوری هماهنگی transition؛ هشدار اختلاف نرخ؛ نود archive برای Router؛ تأمین موجودی گس اوراکل؛ توصیه‌ی heap؛ سرنوشت ورودی مستقیم به distributor؛ بررسی موجودی پیش از ارسال

این جدول آمادگی production را اعلام نمی‌کند.

وضعیت پیاده‌سازی و آزمون

جزء	وضعیت
شش قرارداد ساختاری، SurenSale و ServiceStaking	نوشته‌شده و کامپایل‌شده بدون vialR؛ ServiceStaking فقط پیش‌نویس اولیه است
آزمون خودکار قراردادها (شبکه‌ی شبیه‌ساز Hardhat)	سقف پاداش، حکمرانی نرخ، واجدان رأی، ترتیب آدرس‌ها، فاصله‌ی تغییر سهم و نسخه‌ی ترکیب هیأت پوشش دارند؛ شبیه‌ساز جایگزین Besu واقعی نیست
آزمون روی Besu واقعی	دور چهارم (۲ تا ۴ اکتبر ۲۰۲۶) روی نسخه‌ی فعلی کد و ۲۶/۹/۰ Besu اجرا شد، با شبکه‌ی مستقل برای هر خانواده‌ی آزمون و ثبت شواهد، روی میزبان تکی. مسیر زمان واقعی اجرا نشد و سه ثابت زمانی فقط روی fork آزمایشی فشرده شدند
ابزار ساخت genesis	مشخصات کامل؛ پیاده‌سازی و آزمون باز
RewardRouter	مشخصات کامل (از جمله جدول‌ها و تطبیق crash)؛ پیاده‌سازی و آزمون باز
Verifier، Identity Service، PaymentReporter	مشخصات کامل؛ پیاده‌سازی باز
پروتکل چالش و پاسخ برای سنجش نود (SUR_NODE_CHECK_V1)	طراحی‌شده و هنوز پیاده یا آزموده نشده؛ تا آن زمان فقط حضور در admin_peers سنجیده می‌شود

این جدول آمادگی production را اعلام نمی‌کند.

موارد فنی باز

مورد	وضعیت
genesis gasLimit تولیدی	تعیین نشده؛ هزینه‌ی محاسبه‌ی سقف برای بازه‌ی عقب‌افتاده‌ی دارای k زیاد باید با آن تطبیق داده شود
مسیر رفع «پاداش واقعی بیش از سقف» برای بازه‌ی گذشته	تعریف نشده؛ تغییر نرخ آینده آن را حل نمی‌کند و مسیر دورزدن اضطراری وجود ندارد
اعمال transition نرخ روی Besu	L1 تا L3 روی fork فشرده تأیید شد؛ L4 (زمان واقعی) اجرا نشده. لزوم تغییر فایل و ری‌استارت و رفتار نودهای ناهم‌زمان در دامنه‌ی تک‌میزبان دور چهارم (F05 تا F07) دیده شد و تضمین production نیست
پارامترهای جایگزینی تراکنش RewardRouter	REPLACE_AFTER، MAX_REPLACEMENTS و حداقل افزایش گس باید با txpool واقعی تطبیق یابد
اعداد قواعد تصمیم Verifier	پیش‌نویس؛ آزمون بازپخش با سناریوهای ولیدیتور خاموش، قطع نود پایش، توقف شبکه و ۲۰٪ ساکت
قاعده‌ی انتخاب پیشنهاددهنده‌ی QBFT	محاسبه‌ی نوبت مورد انتظار به نسخه‌ی Besu وابسته است و باید روی شبکه‌ی واقعی راستی‌آزمایی شود
سرویس IP-geolocation و ارائه‌دهنده‌ی پیامک/تلگرام	انتخاب نشده
استثنای تولید بلاک توسط فرد فاقد صلاحیت	گزارش جدا برای تصمیم؛ حذف یا بازتوزیع خودکار ممنوع است

جمع بندی

سور یک شبکه‌ی QBFT روی Besu رسمی است که ولیدیتورها، پاداش و حکمرانی را در شش قرارداد با آدرس ثابت و بدون constructor مدیریت می‌کند؛ این قراردادها مستقیم در alloc می‌نشینند و ساخت genesis با overlay و بررسی از بلاک صفر انجام می‌شود.

ریوارد و فی مستقیم به قرارداد توزیع می‌رسد. چون زنجیره رویدادی ندارد، RewardRouter هر ۲۳ ساعت بازه‌ی بلاک‌ها را از زنجیره محاسبه و با یک تراکنش، زیر سقف پاداش مصوب، تسویه می‌کند.

چهار کلید مستقل در Vault نگهداری می‌شوند؛ Verifier فقط تغییر وضعیت را با هش شواهد ثبت می‌کند و جریمه از مسیر تحویل، اعتراض و رأی مجمع می‌گذرد.

کارهای باز: آزمون نسخه‌ی فعلی روی Besu واقعی، پیاده‌سازی RewardRouter و ابزار genesis و سرویس‌ها، gasLimit تولیدی و مسیر رفع پاداش بیش از سقف.